

5. Security

5.1. Encryption

The process of converting of data into an unreadable format.

Plain Text

Data before applying encryption.

Cipher Text

Data after applying encryption.

Key Encryption:

- To ensure a message is authentic.
- To ensure a message isn't altered during transmission.
- To ensure that only the intended receiver can understand the message.
- To ensure that neither sender nor receiver can deny involvement in the transmission.

Public Key

A public key is shared. It is used to encrypt data so that it can be decrypted with its matching private key.

Private Key

A private key is never shared. It is used to decrypt data that was encrypted with its matching public key.

Symmetric Encryption

A single key is used for both encryption and decryption. The key is shared between the sender and the receiver.

Disadvantages:

- Key has to be exchanged securely.
- Once compromised, it can decrypt sent and received messages.
- Cannot ensure origin or integrity of data.

Asymmetric Encryption

Two keys are used: public key and private key. The message is encrypted with the public key and decrypted using the matching private key.

Symmetric VS Asymmetric

	Symmetric	Asymmetric
Number of Keys	1 key for both	2 keys
Is Key Shared	Yes	Only public key
Security	Less secure as key needs to be shared	More secure
Complexity	Simple	Complex
Speed	Fast	Slow
Key Length	Short	Longer

Security concerns relating to a transmission:

- Confidentiality: Only the intended recipient should be able to decrypt the message.
- Authenticity: Receiver must know who sent the ciphertext.
- Integrity: Message must not be modified during transmission.
- Non-repudiation: Neither the sender nor the receiver should be able to deny involvement in the transmission.

Quantum Cryptography

Quantum Cryptography is used to produce a virtually unbreakable encryption system using the properties of photons.

Advantages:

- More secure.
- Eavesdropping can be detected.
- Longer keys can be used.
- Almost un-hackable.
- Cannot be copied and decrypted at a later time.

Disadvantages:

- Only works for short distances.
- High cost for maintenance and setup.
- Lacks important features like digital signature, certified mail etc.
- High error rate due to being new.
- Polarization of light may be altered during transmission.

5.2. Encryption Protocols

Purpose

- Allow for more security when communicating over the internet as it allows two parties identify and verify each other and communicate confidentially with integrity
- Provides encryption.

Working

- An SSL/TLS connection is initiated between two applications.
- The one initiating it is the client. The one receiving it is the server.
- Session begins with a handshake.
- Server sends its digital certificate and public key
- Client confirms the server's identity.
- Encryption algorithm is agreed upon and the symmetric session keys are generated.

Use Cases

- Transmitting passwords or session cookies.
- Online shopping and banking websites.

Transport Security Layer (TLS)

TLS is a cryptographic protocol designed to provide secure, encrypted communication and authentication over a data network (e.g., HTTPS). It operates between the Transport Layer and the Application Layer of the TCP/IP stack.

- TLS is split into two protocols to manage security
 - **The TLS Handshake protocol:** This layer handles the establishment of the secure connection before any actual application data is transmitted. Authenticates the server and the client and negotiates cryptographic algorithms
 - The client and server agree on the Cipher Suite (encryption algorithms to use).
 - The server sends its Digital Certificate containing its public key.
 - The client verifies the certificate against a trusted Certificate Authority (CA).
 - They securely exchange parameters to generate temporary, symmetric Session Keys used for data encryption.
 - **The TLS Record protocol:** This layer handles the secured transmission of the actual application data during the session. Encrypts, fragments, and verifies data packet integrity.
 - Takes data from the application layer and breaks it into manageable blocks.
 - Compresses the data (optional).
 - Applies a Message Authentication Code (MAC) or hash to ensure data integrity (prevents tampering).
 - Encrypts the block using the symmetric session key established during the Handshake.

SSL	TSL
Connects directly via port numbers	Connects via a standard port and then upgrades to secure mode via explicit commands
Closes the connection immediately without sending warning codes	Uses specific, encrypted alert codes to explain network or certificate errors
Supports older, weak encryption algorithms (e.g., DES, RC4)making it vulnerable to attacks	Supports strong, modern cryptographic algorithms (e.g., AES, ChaCha20).

Table: Differences between SSL and TSL

Session Caching

The TLS handshake is computationally expensive and time-consuming because it uses asymmetric cryptography which requires heavy CPU processing to set up keys.

- **Session Caching:** A performance optimization mechanism where the server and client store (cache) the parameters of a recently established TLS session for a specific period of time.
- **How it works:**
 - If a user reconnects to the server shortly after disconnecting, the client passes a cached Session ID or Session Ticket back to the server.
 - If the server recognizes it, it skips the full asymmetric handshake and immediately resumes encryption using the pre-existing session keys.
- **Why it is needed:**
 - Reduces Latency:Decreases network round-trips, making page loads faster.
 - Saves CPU Resources: Minimizes the computational overhead on the server, allowing it to handle thousands of concurrent secure connections easily.

5.3. Digital Certification

Digital Certificate

1. An organization sends a request to Certificate Authority (CA).
2. They send their public key, information to prove their identity as well as any additional information required by the CA.
3. The CA then verifies their identity and then issues the certificate after encrypting it with their private key.

Digital Signature

The message is put through a hashing algorithm to produce the digest. This is then encrypted with the sender's private key. This is the digital signature.

When the message is received, the digital signature is then decrypted with the sender's public key. The decrypted message is put through the same hashing function to output a digest. The two digests are compared and if they are the same then the message was not tampered with.